# Customer Churn Prediction

Complete end-to-end Python + Machine Learning + GitHub project guide

## 1. Project Overview

Objective: predict whether a subscription customer is likely to churn using historical customer demographics, service usage, billing behavior, support interactions and account activity. This is a binary classification problem. The final system outputs a churn probability and a business-friendly risk label.

## 2. Technology Stack

Python, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, Imbalanced-learn (SMOTE), Joblib and optionally XGBoost/LightGBM. The project is designed to run in Jupyter Notebook or VS Code.

## 3. GitHub Project Structure

```
customer-churn-prediction/
■■■ data/
■   ■■■ raw/
■   ■■■ processed/
■■■ notebooks/
■   ■■■ customer_churn_prediction.ipynb
■■■ src/
■   ■■■ data_preprocessing.py
■   ■■■ train_model.py
■   ■■■ predict.py
■■■ models/
■   ■■■ churn_pipeline.joblib
■■■ outputs/
■   ■■■ confusion_matrix.png
■   ■■■ feature_importance.png
■■■ requirements.txt
■■■ README.md
```

## 4. Dataset

Recommended public datasets include the IBM Telco Customer Churn dataset or another subscription dataset containing tenure, monthly charges, contract type, payment method, service usage and churn. Do not commit sensitive real customer information to GitHub. Place the downloaded CSV in data/raw/.

## 5. Install Dependencies

```
pip install pandas numpy matplotlib seaborn scikit-learn imbalanced-learn joblib
# Optional:
pip install lightgbm
```

## 6. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             classification_report, RocCurveDisplay)
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
```

```
import joblib
```

## 7. Load and Inspect Data

```
df = pd.read_csv('data/raw/customer_churn.csv')
print(df.shape)
print(df.head())
print(df.info())
print(df.isnull().sum())
print(df['Churn'].value_counts())
```

## 8. Data Cleaning

```
# Example: convert TotalCharges if it is stored as text
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# Remove identifier that should not drive the prediction
if 'customerID' in df.columns:
    df = df.drop(columns=['customerID'])

# Standardize target
if df['Churn'].dtype == 'object':
    df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})

print(df.isnull().sum())
```

## 9. Exploratory Data Analysis

```
sns.countplot(data=df, x='Churn')
plt.title('Churn Distribution')
plt.show()

sns.boxplot(data=df, x='Churn', y='MonthlyCharges')
plt.title('Monthly Charges vs Churn')
plt.show()

sns.histplot(data=df, x='tenure', hue='Churn', kde=True, element='step')
plt.title('Tenure Distribution by Churn')
plt.show()
```

## 10. Feature / Target Split

```
X = df.drop(columns=['Churn'])
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

numeric_features = X.select_dtypes(include=['int64','float64']).columns.tolist()
categorical_features = X.select_dtypes(include=['object','category','bool']).columns.tolist()
```

## 11. Preprocessing

```
numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_pipe, numeric_features),
    ('cat', categorical_pipe, categorical_features)
])
```

## 12. Logistic Regression + SMOTE

SMOTE should be applied only to the training data. Putting preprocessing and SMOTE inside the training pipeline prevents test-data leakage.

```
logistic_model = ImbPipeline([
    ('preprocess', preprocessor),
    ('smote', SMOTE(random_state=42)),
    ('model', LogisticRegression(max_iter=2000, class_weight=None))
])

logistic_model.fit(X_train, y_train)
y_pred = logistic_model.predict(X_test)
y_prob = logistic_model.predict_proba(X_test)[:, 1]
```

# 13. Evaluation

```
def evaluate_model(model, X_test, y_test):
    pred = model.predict(X_test)
    prob = model.predict_proba(X_test)[:, 1]
    print('Accuracy :', round(accuracy_score(y_test, pred), 4))
    print('Precision:', round(precision_score(y_test, pred), 4))
    print('Recall   :', round(recall_score(y_test, pred), 4))
    print('F1 Score :', round(f1_score(y_test, pred), 4))
    print('ROC-AUC  :', round(roc_auc_score(y_test, prob), 4))
    print('\nClassification Report:\n', classification_report(y_test, pred))
    print('Confusion Matrix:\n', confusion_matrix(y_test, pred))

evaluate_model(logistic_model, X_test, y_test)
RocCurveDisplay.from_estimator(logistic_model, X_test, y_test)
plt.show()
```

# 14. Decision Tree Model

```
tree_model = ImbPipeline([
    ('preprocess', preprocessor),
    ('smote', SMOTE(random_state=42)),
    ('model', DecisionTreeClassifier(max_depth=6, random_state=42))
])
tree_model.fit(X_train, y_train)
evaluate_model(tree_model, X_test, y_test)
```

# 15. Hyperparameter Tuning

```
params = {
    'model__max_depth': [3, 5, 7, 10, None],
    'model__min_samples_split': [2, 5, 10],
    'model__min_samples_leaf': [1, 2, 5]
}

grid = GridSearchCV(tree_model, params, scoring='roc_auc', cv=5, n_jobs=-1)
grid.fit(X_train, y_train)
print(grid.best_params_)
print(grid.best_score_)
best_churn_model = grid.best_estimator_
```

# 16. Save Model

```
joblib.dump(best_churn_model, 'models/churn_pipeline.joblib')
print('Model saved successfully.')
```

# 17. Predict a New Customer

```
model = joblib.load('models/churn_pipeline.joblib')

new_customer = pd.DataFrame([{
    'gender': 'Female',
    'SeniorCitizen': 0,
    'Partner': 'Yes',
    'Dependents': 'No',
    'tenure': 8,
    'PhoneService': 'Yes',
    'MultipleLines': 'No',
    'InternetService': 'Fiber optic',
    'OnlineSecurity': 'No',
    'OnlineBackup': 'No',
    'DeviceProtection': 'No',
    'TechSupport': 'No',
    'StreamingTV': 'Yes',
    'StreamingMovies': 'Yes',
    'Contract': 'Month-to-month',
    'PaperlessBilling': 'Yes',
```

```
    'PaymentMethod': 'Electronic check',
    'MonthlyCharges': 85.0,
    'TotalCharges': 680.0
}])

probability = model.predict_proba(new_customer)[0, 1]
prediction = int(probability >= 0.50)
print('Churn probability:', round(probability, 3))
print('Prediction:', 'Likely to churn' if prediction else 'Likely to stay')
```

## 18. Business Insights

The project can be extended into a retention dashboard. High-risk customers can be grouped by contract type, tenure, monthly charges, support usage and payment behavior. The model should support—not replace—business decisions about retention offers.

## 19. requirements.txt

```
pandas
numpy
matplotlib
seaborn
scikit-learn
imbalanced-learn
joblib
jupyter
```

## 20. README.md Template

```
# Customer Churn Prediction

## Problem
Predict customers likely to leave a subscription service.

## Workflow
Data collection -> cleaning -> EDA -> preprocessing -> SMOTE -> model training -> evaluation -> prediction.

## Models
- Logistic Regression
- Decision Tree
- Hyperparameter tuning

## Metrics
Accuracy, Precision, Recall, F1 and ROC-AUC.

## Run
1. Create a virtual environment.
2. Install requirements.
3. Put the CSV in data/raw/.
4. Run the notebook.
5. The trained model is saved in models/.

## Skills Demonstrated
Python, Pandas, EDA, classification, imbalanced learning, model evaluation and deployment-ready preprocessing.
```

## 21. Interview Explanation

A strong interview explanation is: “I built a binary classification system to identify customers at risk of churn. I cleaned and explored historical customer data, separated numeric and categorical variables, used one-hot encoding and scaling, handled class imbalance with SMOTE only on the training pipeline, compared Logistic Regression and Decision Tree models, evaluated them using precision, recall, F1 and ROC-AUC, tuned the tree, and saved the final preprocessing-plus-model pipeline with Joblib for future predictions.”